

CS481 HW1

Tomas Salaz

February , 2026

Question 1

In CS341, we learned about Virtual Memory (VM) and Physical Memory (PM). To my understanding, when we call `fork()` in `processes.cpp`, the child process gets a copy of the parents VM space, but it gets its own PM space, so the same VM addresses are being mapped to different PM addresses.

In the `example/example.c` file, the global counter is printing the VM address which is the same for all the children processes, but the value of the counter is changed in the PM address which is why the values for the different processes is updated independently.

Question 2

To change the ordering for the parent and child processes, we would need to change ordering of the `wait()` and the `parent()` calls in the code (in my code, lines 25-27).

This would allow the `parent()` method to run right away, then wait for the other children processes.

Question 3

$$T_{turnaround} = T_{completion} - T_{arrival}$$

$$T_{response} = T_{firstrun} - T_{arrival}$$

3.1

Assuming all jobs arrive at $t = 0$

Job0 would run first for 4s, then *Job1* for 6s (10s total), then *Job2* for 3s, for a total time of 13s.

$$T_{turnaround} = (4 + 10 + 13)/3 = 27/3 = 9$$

Job0 starts at 0s, then *Job1* starts at 4s, then *Job2* starts at 10s.

$$T_{response} = (0 + 4 + 10)/3 = 14/3 = 4.6$$

3.2

Assuming all jobs arrive at $t = 0$

Job0 runs completely by priority, regardless of time slice, for 4s. *Job1* runs for 2s (4s remain), then *Job2* for 2s (1s remain), then *Job1* again for 2s (2s remain), then *Job2* runs for 1s (is done at 11s), then *Job1* runs for the last 2s (done at 13s).

$$T_{turnaround} = (4 + 11 + 13)/3 = 28/3 = 9.3$$

Following the same timeline, *Job0* starts at 0s, *Job1* starts at 4s, and *Job2* starts at 6s

$$T_{response} = (0 + 4 + 6)/3 = 10/3 = 3.3$$